

Short Papers

kvTZ: TrustZone Virtualization for Commodity Arm-Based Platforms

Chun-Yen Lin  and Shih-Wei Li 

Abstract—Arm TrustZone technology provides hardware features to enable the deployment of security-critical software in trusted execution environments (TEEs). Although TrustZone is widely deployed on physical hardware, it is unavailable to increasingly deployed virtual machines (VMs) running on commodity Arm platforms. These VMs cannot leverage TrustZone’s security features, such as secure boot, or deploy trusted applications to secure their systems. To address this limitation, we propose a new design, called kvTZ, that extends commodity hypervisors to expose a virtualized TrustZone to VMs. kvTZ introduces exception-level multiplexing, a novel technique that enables native execution of TrustZone software in the VM environment on the existing Arm hardware. We prototyped kvTZ by extending KVM implementations, including the mainline Linux and Google’s Android Linux for pKVM, to support legacy and confidential VMs. kvTZ supports OP-TEE, a de facto open-source TEE for Arm TrustZone. For the first time, we enabled OP-TEE’s entire software stack, which encompasses trusted applications (TAs), the kernel, and trusted firmware, to run in a virtualized TrustZone. We show that kvTZ achieves performance efficiency and outperforms the full software emulation-based solution.

Index Terms—Operating systems, security, virtualization.

I. INTRODUCTION

Arm imposes hardware mechanisms to isolate hardware resources (e.g., memory). TrustZone [1] is a security extension for Arm System-On-Chips (SoCs) to enable two isolated execution environments: a secure world to execute a Trusted Execution Environment (TEE) and a normal world to execute a Rich Execution Environment (REE). The TEE runs a trusted kernel that manages trusted applications (TAs), while the REE hosts a commodity OS, such as Linux, and their applications. Systems deployed in practice isolate security-critical TAs, such as those for cryptographic applications and key management in TEEs, from potentially compromised REEs. TrustZone imposes isolation mechanisms to seal TEEs’ execution context and resources (e.g., memory and I/O device accesses).

Arm’s TrustZone extensions introduced a new monitor mode (EL3), a higher-privileged CPU execution level. In the Arm ecosystem, trusted firmware (e.g., TF-A [2]) that runs in EL3 includes a secure monitor. The monitor is a gatekeeper, managing transitions between the normal and secure world. Additionally, the trusted firmware provides essential security features, including secure boot, which cryptographically verifies the integrity of software components before execution, maintaining a root of trust throughout the boot sequence.

Arm processors have been increasingly deployed [3], [4]. As virtual machines (VMs) have become essential in modern computing infrastructures, commodity hypervisors, such as KVM [5], have been

extended to host Arm-based guest VMs. To enhance security, KVM now supports confidential computing, preventing a compromised hypervisor from accessing or tampering with confidential VM (CVM) data and execution. To support CVMs, Arm has recently updated the architecture with Confidential Compute Architecture (CCA) extensions. Since no CCA hardware is available, developers have extended KVM to embed a small pKVM [6] hypervisor that utilizes features from existing Arm hardware to protect CVMs.

Despite the increasing adoption of Arm TrustZone technology, no existing commodity hypervisors expose its capabilities to VMs or CVMs, which prevents users from leveraging TrustZone to deploy trusted applications or utilize its firmware-based security features. Moreover, virtualized TrustZone support is crucial for facilitating the software development and testing workflows of TEEs, such as fuzzing, by eliminating hardware dependencies. While QEMU employs binary translation to run TrustZone-enabled VMs in user space, this approach incurs significant performance overhead. With the growing prevalence of Arm-based hardware, enabling efficient native code execution for virtualized TrustZone is essential.

vTZ [7] pioneered the introduction of a design that exposes virtualized TrustZone to VMs on Xen. However, vTZ suffers from several key limitations that impede its adoption and functionality: (1) *Limited TrustZone Stack Support*. vTZ enables the execution of TAs and trusted kernels within TEEs in VMs. It does not virtualize the EL3 environment, preventing VMs from running TrustZone’s trusted firmware and fully leveraging its security features. (2) *Incompatible VM Abstractions*. vTZ focuses on protecting a VM’s TEE from a malicious hypervisor. vTZ is incompatible with legacy VM use cases where users fully trust the hypervisor. Furthermore, vTZ does not isolate the VM’s REE from the hypervisor. This makes it incompatible with CVMs, which require full VM protection. (3) *Deployment Challenges & Hypervisor Incompatibility*. vTZ requires installing a custom EL3 monitor into the physical TrustZone, effectively necessitating a replacement of the Arm platform’s existing firmware. This design creates a significant barrier to practical deployment and prevents adoption by commodity hypervisors, such as KVM.

To address these limitations, this work introduces kvTZ, a novel design that extends Arm commodity hypervisors to enable the execution of virtualized TrustZone environments. For (1), kvTZ allows the execution of a full-stack TrustZone software, including the TEE and trusted firmware. kvTZ introduces *exception-level (EL) multiplexing* to enable, for the first time, a virtual EL3 environment. This technique multiplexes the VM’s TEE software and trusted firmware onto the hardware’s normal world, retaining performance efficiency and portability. This allows most of the TrustZone software to execute natively in the VM. For operations that require hypervisor intervention, such as those that access sensitive EL3 registers, kvTZ employs trap-and-emulate to ensure these sensitive operations are handled securely.

To tackle (2), kvTZ provides an extensible design to support both the legacy VM and CVM abstractions. For the latter, kvTZ matches

Received 21 March 2025; revised 29 July 2025; accepted 29 August 2025. Date of publication 3 September 2025; date of current version 14 January 2026. This work was supported by the National Science and Technology Council of Taiwan under Grant 111-2628-E-002-022-, Grant 112-2628-E-002-027-, Grant 113-2628-E-002-030-, and Grant 114-2628-E-002-001-. (Corresponding author: Shih-Wei Li.)

The authors are with National Taiwan University, Taipei 10617, Taiwan (e-mail: r11922153@csie.ntu.edu.tw; shihwei@csie.ntu.edu.tw).

Digital Object Identifier 10.1109/TDSC.2025.3605807

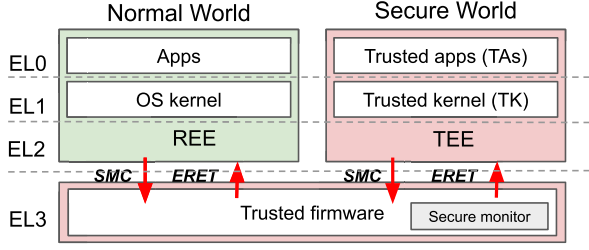


Fig. 1. Arm TrustZone architecture.

the full CVM protection, which isolates not only the TrustZone but also the REE from the hypervisor. Regarding (3), unlike vTZ, kvTZ requires self-contained modifications to the hypervisor codebase. This design eliminates the dependency on physical TrustZone hardware, enabling kvTZ to be seamlessly deployed on commodity hypervisors, such as KVM. This approach not only enables existing VMs on standard Arm platforms to easily adopt TrustZone features but also provides developers with a much-needed, flexible, and hardware-independent testing environment.

We demonstrate kvTZ's practicality and portability in supporting commodity hypervisors. We extended two KVM variants to prototype kvTZ to enable virtualized TrustZone in both VMs and CVMs running on an Arm server and embedded device for the first time: KVM in the mainline Linux and Android Linux with pKVM support [8]. These prototypes offer comprehensive support for OP-TEE [9], the de facto open-source TEE solution for Arm, of its entire software stack, including the trusted firmware, trusted OS kernel, and trusted applications. The kvTZ prototypes deliver an order-of-magnitude performance improvement over the full software emulation-based solution and achieve less than 5% overhead in the best case compared to bare-metal execution in TEE application workloads.

II. BACKGROUND

A. Arm TrustZone

Arm extends the CPUs to include two security states to support TrustZone [1]: secure and non-secure (normal) worlds. Both worlds provide exception levels for EL0 (user mode) and EL1 (kernel mode). The secure state also supports EL3 (monitor mode) to run the trusted firmware. TrustZone's monitor, included in the firmware, multiplexes the security states on a single CPU to support two execution environments: secure and normal world. In a TrustZone-enabled system, the REE runs in the normal world, and the TEE runs in the secure world. Fig. 1 shows the TrustZone architecture. Since Arm v8.4, the secure world has also supported an EL2 mode to execute a hypervisor. However, commodity systems such as OP-TEE [9] have yet to integrate such hypervisor capabilities.

Arm provides banked EL3 system registers, allowing software that manages world transitions to run in an isolated environment. Arm provides a special instruction called the secure monitor call (SMC) [10] to the TEE and REE to invoke the secure monitor and trigger a world switch. Arm provides an exception return instruction, ERET, for EL3 to return to a lower-privileged mode. ERET pops the execution context, including the PC and CPU states, to the hardware.

In addition to providing two execution environments, TrustZone offers vendors optional hardware device extensions, such as secure memory and peripherals, like UART and GPIO.

OP-TEE [9]: OP-TEE is a widely-used open-source TEE implementation that supports an Arm TrustZone environment. The OP-TEE

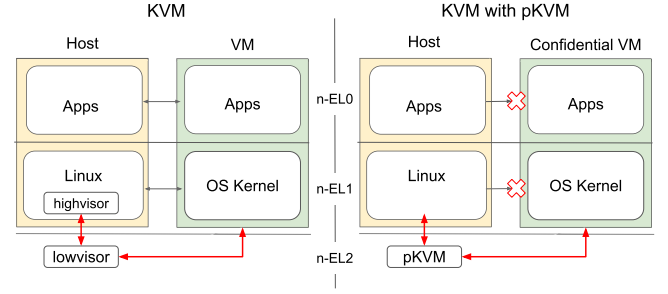


Fig. 2. KVM architecture for Arm.

kernel and its hosted trusted applications run in EL1 and EL0 of the secure world. OP-TEE integrates a secure monitor implemented by Arm's Trusted Firmware (TF-A) [2]. The TF-A runs in EL3 to initialize the system and installs the trusted OP-TEE kernel into secure EL1. The OP-TEE kernel then loads the TAs to secure EL0. The TF-A and its integrated boot software (e.g., the u-boot bootloader) provide a secure boot, ensuring the authenticated REE OS kernel is loaded.

To execute a TA, the REE's OS kernel first invokes SMC, which causes the CPU to switch to EL3. TF-A's monitor handles the SMC by context switching to the TEE's trusted kernel. Specifically, it saves the normal world's EL1 system registers from the hardware and restores the secure world's registers. It then invokes ERET to enter the TEE kernel in secure EL1, which subsequently invokes the appropriate TA. When the TA returns, the TEE kernel invokes another SMC to enter EL3. This time, the monitor saves the secure world's EL1 system registers and loads the normal world states to the hardware. Finally, the monitor then invokes ERET to return to the REE kernel in normal EL1.

B. KVM for Arm

KVM [11] leverages existing Linux functionality to support virtual machines while integrating with QEMU [12] for device virtualization and VM management. When co-working with KVM, QEMU's Tiny Code Generator (TCG) [13] binary translator, which supports running fully emulated VM code at user space, is disabled. KVM for Arm utilizes hardware support, Arm's Virtualization Extensions (VE) [14] to run unmodified OS kernels and applications natively in a virtualized execution context in the normal world.

Arm VE introduces EL2, a higher-privileged CPU mode than EL1 and EL0, to execute hypervisors in an isolated environment. KVM for Arm [15] employs split-mode virtualization that leverages Arm VE. It runs a highvisor with the Linux host in EL1 to support comprehensive virtualization functionalities and a lowvisor in EL2 to manage VE features. The lowvisor runs the VM's kernel and applications in EL1 and EL0. As shown in Fig. 2, the lowvisor interposes VM enters and exits to multiplex the EL1 execution context on the hardware between the host and VMs. Arm VE extends the MMU to walk guest page tables (maintained by VMs) and stage 2 page tables (S2PTs) for address translation to support memory virtualization. The latter translates the output of the guest page tables to the machine's physical address.

Confidential computing has emerged as a crucial technology for protecting CVMs on multi-tenant environments from compromised hypervisors. Developers from Google have recently extended KVM to integrate a secure module called pKVM [6], which serves as the Trusted Computing Base (TCB). pKVM leverages Arm VE to support CVMs. pKVM ensures that the KVM host, including the entire Linux, cannot access CVMs' private memory and CPU states to protect their confidentiality and integrity. As shown in Fig. 2, pKVM runs in EL2

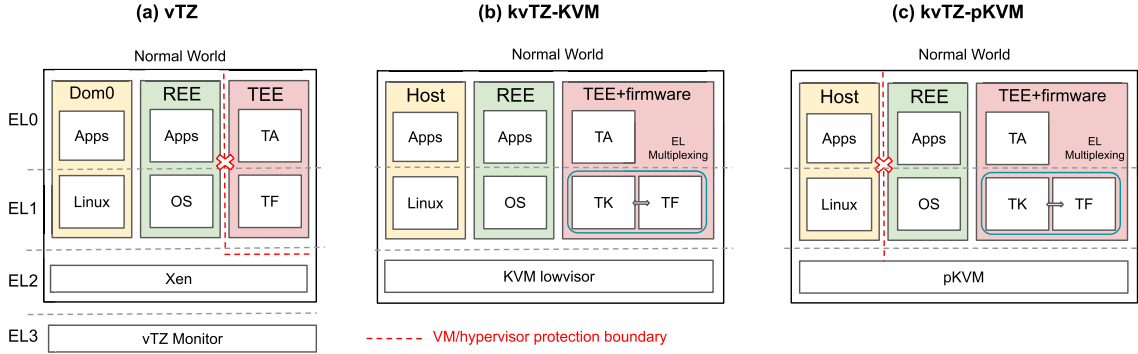


Fig. 3. TrustZone virtualization solutions. *TK*: TEE kernel. *TF*: TrustZone’s secure firmware. vTZ protects the VM’s TEE from Xen and Dom0. kvTZ-KVM does not isolate the VM from KVM and the host; it inherits the security boundaries of KVM. kvTZ-pKVM protects the entire VM (CVM), including its REE, TEE, and firmware from KVM and the host.

to interpose VM exits/entries to protect CPU states. pKVM uses Arm’s S2PTs to restrict the host’s access to CVM memory. Arm CCA focuses on providing similar CVM protection to pKVM but integrates hardware extensions.

III. DESIGN

A. Limitations of Past Work

QEMU supports Arm TrustZone by emulating it through binary translation. This approach incurs significant performance overhead. vTZ [7] extends Xen to run TEEs natively in VMs on Arm hardware. As shown in Fig. 3, vTZ integrates a trusted EL3 monitor that enables TrustZone virtualization capabilities. vTZ executes both the VM’s TEE and REE in the hardware’s normal world. The monitor interposes switches between the two to multiplex their execution context on the hardware. We identify key limitations of vTZ.

1) *Limited TrustZone Stack Support*: As shown in Fig. 3, vTZ does not virtualize the EL3 environment. vTZ focuses on virtualizing TrustZone environments to run TEE software: the trusted kernel and applications. Consequently, VMs cannot execute the trusted firmware and utilize features, such as secure boot, for enhanced security.

2) *Incompatible VM Abstractions*: vTZ’s design prevents a malicious hypervisor from compromising the virtualized TEE. It does not support use cases where users fully trust the hypervisor. Consequently, vTZ cannot be used for legacy VM deployments, including general workload hosting, or in development and testing scenarios where the hypervisor is assumed to be benign. On the other hand, vTZ’s design is incompatible with CVMs, where not only the TEE, but the entire virtualized environment, including the TEE, REE, and trusted firmware, must be protected from the hypervisor.

3) *Deployment Challenges & Hypervisor Incompatibility*: Fig. 3 shows that vTZ’s monitor executes in the hardware’s EL3—the same privilege level used by Arm’s firmware. This physical TrustZone dependency complicates real-world deployment. It mandates users to install a customized TrustZone monitor on their hardware to replace the existing platform firmware. It also makes vTZ inherently incompatible with commodity hypervisors, like KVM, limiting adoption and posing substantial hurdles upstream with the mainline code.

B. Threat Model

We assume a guest REE, TEE, and trusted firmware might contain bugs. kvTZ cannot prevent their bugs or defend against guests that intentionally bypass TrustZone-enforced isolation. We consider an adversary

with varying capabilities depending on the kvTZ deployment. For kvTZ deployed to host legacy VMs, we consider the entire hypervisor part of the TCB. In contrast, for CVM deployments, we consider the hypervisor potentially adversarial and attempt to tamper with CVMs; the secure module, integrating kvTZ’s extensions, serves the TCB and preserves CVM protection. We assume kvTZ’s TCB and the underlying Arm hardware are bug-free. We exclude side-channel, hardware, and physical attacks from the threat model.

C. kvTZ Design

kvTZ tackles vTZ’s limitations. For (1), kvTZ virtualizes the entire secure world, including secure EL3, EL1, and EL0, allowing the execution of the full TrustZone software stack, including the trusted firmware and TEE, enabling users to fully leverage TrustZone’s security guarantees. kvTZ ensures that the virtualized TrustZone retains the security requirements of a physical implementation. kvTZ isolates the secure world’s execution contexts and resources, including secure memory and I/O devices, from the normal world. It also isolates the secure EL3 execution context from secure EL1.

To address (2), kvTZ customizes hypervisor extensions tailored to the security demands of the specific implementation. For legacy unprotected VMs, kvTZ extends the hypervisor, which constitutes a TCB, to provide TrustZone virtualization. For CVMs, kvTZ extends the secure module (e.g., pKVM) integrated with CVM frameworks to provide TrustZone functionalities that require access to private CVM execution states—specifically world-switching operations and page table management. kvTZ ensures that the hypervisor cannot access both the CVM’s secure and normal worlds’ CPU and memory. Meanwhile, kvTZ strategically offloads other TrustZone virtualization functions, such as I/O virtualization, to the hypervisor, carefully limiting these operations to those requiring no access to protected CVM states. This architecture maintains the security guarantees of CVMs while enabling full TrustZone functionality.

To address (3), kvTZ employs a self-contained approach incorporating extensions to the hypervisor codebase. Eliminating dependency on physical TrustZone hardware simplifies deployment across diverse Arm-based platforms. This design approach enables kvTZ to deliver better portability than vTZ, exposing virtualized TrustZone to VMs running on hardware without TrustZone features, enabling efficient deployment.

We detail kvTZ’s design as follows.

1) *CPU Virtualization*: In the secure world, software from the TrustZone-based TEE executes in EL0 and EL1, while the firmware

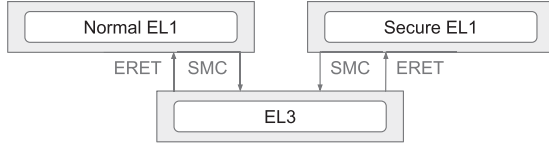


Fig. 4. Execution context multiplexed to normal EL1.

runs in EL3. Therefore, kvTZ must faithfully provide a virtualized TrustZone environment to the VM that mirrors the hardware, ensuring that the software executing across these CPU modes can operate correctly.

TrustZone software configures Arm’s system registers to establish the execution environment. For instance, the trusted kernel and secure firmware, respectively, manage the SCTLR register banked in secure EL1 and EL3 to configure the virtual memory status, such as paging, for EL0/EL1 and EL3. kvTZ must ensure that when a VM executes the TEE and firmware, its instructions that read from or write to system registers are correctly virtualized and reflect its execution states.

Hypervisors like KVM leverage Arm VE and execute VMs in the hardware’s normal world. Therefore, our approach ensures that TrustZone virtualization remains compatible with this established paradigm. kvTZ addresses unique challenges and runs the VM’s REE and TEE, along with its firmware, entirely within the hardware’s normal world.

kvTZ introduced *Exception Level (EL) multiplexing*, which exploits the following observations. It virtualizes the TrustZone environment on the hardware’s normal world and enables the execution of a complete TrustZone software stack.

- *O1*: Arm provides the same EL1 system registers in normal and secure worlds.
- *O2*: System registers banked in both EL1 and EL3 have the same width and format.

Execution context multiplex: kvTZ builds on *O1* to load the VM’s secure world EL1 system registers on the hardware’s normal world EL1. On the other hand, for system registers banked in secure EL3 and EL1, kvTZ exploits *O2* to load the states of the virtual secure EL3 registers into the hardware’s EL1 system registers. kvTZ aims to preserve TrustZone’s security. It isolates the execution context of (1) the secure world from the normal world and (2) the virtualized EL3 from EL0 and EL1. For instance, kvTZ ensures that the VM’s trusted kernel cannot access EL3’s registers.

To create a virtual EL3 and EL1 environment based on the physical EL1 environment, kvTZ must context-switch the EL1 register states whenever the VM changes the exception level between EL1 and EL3. Fig. 4 demonstrates the transitions of the VM’s execution context loaded to the hardware. In the context of TrustZone virtualization, kvTZ intercepts VMs/CVMs’ invocation of two instructions: SMC and ERET. SMC causes a change from either normal or secure EL1 to EL3; executing ERET causes a change from EL3 to EL1. kvTZ traps the VMs’ execution of the SMC and ERET instructions, such that they can be emulated for the intended behaviors. kvTZ context switches the respective virtual states to the physical hardware.

For SMC, the emulation process involves saving the program counter to virtual link register: ELR_EL3, and processor state to virtual saved state register, SPSR_EL3, then setting the Program Counter (PC) to virtual VBAR_EL3 (exception vector base register) with the appropriate offset to redirect execution to the EL3 exception vector table (maintained by the monitor). Finally, the hypervisor saves the virtual EL1 system registers from the hardware, restores virtual EL3 registers to hardware EL1, and invokes the trusted firmware in the VM.

For ERET, kvTZ first saves the virtual EL3 states loaded to the hardware to memory and then restores virtual EL1 states previously saved during SMC emulation to the hardware. kvTZ retrieves the PC and processor state to restore the hardware for EL1 from the virtual registers: ELR_EL3 and SPSR_EL3.

Register access redirection: TrustZone TEE’s trusted kernel accesses secure EL1 system registers. Due to *O1*, we can execute the kernel in the physical EL1 mode without redirection efforts. For the trusted firmware, we exploit *O2* and redirect instructions that access EL3 system registers to instead access the corresponding EL1 system registers that we load onto the hardware. Since Arm VE does not automatically redirect access to EL3 system registers to their banked EL1 counterparts, *EL multiplexing* employs a paravirtualization-based approach. Specifically, we replace instructions from TrustZone’s secure firmware that access EL3 registers with those that access their EL1 equivalent banked registers. As an example, we *paravirtualize* the trusted firmware to replace its MSR instruction, which accesses system register SPSR_EL3, with one that accesses SPSR_EL1.

Trap-and-emulate Some system registers exist exclusively at EL3, such as SCR_EL3 and MDCR_EL3 (Monitor Debug Configuration Register), and lack EL1 counterparts. kvTZ employs the *trap-and-emulate* approach to intercept the firmware’s accesses to these EL3-exclusive registers and emulate them. We ensure the VM’s intended changes to these registers do not affect the physical hardware. Reflecting these states on the hardware’s EL3 registers could allow VMs to hijack the system. Instead, we ensure the operations are conducted only against virtual registers in memory.

Aside from EL3 system registers, the trusted firmware also accesses EL1 system registers. As mentioned in Section II-A, such access is involved when the firmware context switches between the TEE’s and REE’s EL1 states. Since kvTZ loads the VM firmware’s virtual EL3 states on hardware’s EL1, granting it direct access to EL1 system registers would result in erroneously operating on its own virtual EL3 state rather than the intended virtual EL1 state. kvTZ employs trap-and-emulate to intercept these problematic instructions that the firmware uses and emulate the operations against virtual EL1 states.

2) Memory Virtualization: In a TrustZone environment, the firmware provides hardware-enforced isolation of security-critical data from normal world access. This protected memory region occupies a dedicated physical address space exclusively accessible to secure world components, with the isolation boundary enforced by TrustZone’s hardware memory access control mechanisms. Secure memory is essential to enable the proper execution of TrustZone components like the firmware in an isolated environment. kvTZ virtualizes TrustZone’s secure memory partition. To isolate accesses to secure memory, kvTZ creates two sets of S2PT for a TrustZone-enabled VM, *normal-VTTBR* and *secure-VTTBR*, respectively, used when executing the virtual normal and secure world. We ensure that *secure-VTTBR* maps the secure and normal memory, but *normal-VTTBR* only maps the normal memory.

3) I/O Virtualization: kvTZ leverages the existing virtual TrustZone I/O device abstractions in virtual machine monitors (VMMs), such as QEMU. Arm devices use memory-mapped I/Os (MMIOs) to I/O devices. kvTZ employs a trap-and-emulate strategy to virtualize TrustZone’s I/Os. kvTZ unmaps the address regions for secure devices in the VM’s *secure-VTTBR*. Therefore, MMIO accesses made by the virtualized TrustZone to these devices cause an exit to the hypervisor. The respective virtual device in the VMM is later invoked to handle the MMIO access. kvTZ ensures that secure devices are inaccessible to the virtualized normal world. Arm TrustZone supports secure interrupts, which are generated by secure devices and delivered to the secure world. Our design multiplexes Arm’s virtual generic interrupt controller (vGIC) between the virtualized secure and normal world. We introduce

a secure interrupt dispatch that validates the interrupt's source and raises secure interrupts in the secure world's vGIC.

IV. IMPLEMENTATION

We prototyped kvTZ to expose a virtualized TrustZone environment to VMs and confidential VMs. Our KVM prototypes support two Arm hardware: Arm AVA Developer Platform [16] and Raspberry Pi 3 Model B, showing kvTZ's portability. We extended three KVM versions to implement kvTZ: (1) KVM in mainline Linux v5.15, (2) KVM with pKVM support from the Android Linux repository [8], and (3) KVM in Raspberry Pi Linux v5.15 (rpi-5.15.y) [26]. We added and modified roughly 440 LOC to the mainline and Raspberry Pi Linux, and 507 LOC to the pKVM Linux. We extended QEMU v8.0.0 with 432 LOC to utilize its virtual device support for VMs running on the extended KVM. Our implementation leverages QEMU's TrustZone abstraction with secure memory and virtual devices, including a secure UART and GPIO device.

kvTZ requires no REE app, TA, and TEE API changes. We paravirtualized the OP-TEE kernel and trusted firmware to execute in virtualized environments. We added and modified 483 LOC to the OP-TEE kernel and incorporated trusted firmware, TF-A (v2.9), both in the OP-TEE framework v4.0.0. We added 10 LOC to the guest Linux kernel. The modifications are required to support EL multiplexing and enable trap-and-emulate for sensitive instructions. For the former, we modified the MSR and MRS instructions from TF-A to access EL1 system registers instead of EL3 registers. For trap-and-emulate, we replaced instructions with hypercalls to induce VM exits. This includes: (1) MRS and MSR instructions used by TF-A to access EL3-exclusive and EL1 system registers (the latter is used during TEE/REE switches), (2) SMC instructions utilized by Linux and OP-TEE kernels, (3) ERET instructions used by TF-A, and (4) HLT instructions used by TF-A and the OP-TEE kernel; TrustZone software, such as TF-A, uses HLT to load boot images from the host.

We extended KVM and pKVM's TCB to handle virtual EL1 and EL3 transitions and multiplex virtual states onto the hardware. Our implementations ensure state isolation upon transitioning between REEs, TEEs, and trusted firmware (as shown in Fig. 4). To support EL multiplexing, it context switches the hardware EL1 system registers, including the page table base register, TTBR, with the value designated for the target virtual context. To ensure address space isolation, we flush all TLB entries in the TTBR switch. As a future enhancement, we plan to explore the incorporation of TLB tagging to optimize performance.

In the initial KVM's kvTZ implementation, we extended KVM's hypercall handling interface to emulate virtual TrustZones' system register accesses. However, this incurred significant overhead due to costly KVM context switches to and from the host. Each such switch necessitated memory accesses for multiplexing EL1 states on the hardware. To eliminate these context switches, we introduced a *system register optimization* that handles instruction emulation directly within the KVM lowvisor at EL2, allowing a direct return to the VM.

V. EVALUATION

A. Performance Evaluation

We evaluated our KVM implementations on the AVA Platform and Raspberry Pi 3 Model B (denoted *RPI3B*). The former features a 32-core Arm Neoverse N1 CPU, 32 GB of RAM, and 128 GB of NVMe M.2 storage, while the latter has a Quad-Core Broadcom BCM2837 CPU, 1 GB of RAM, and a 64 GB SanDisk Ultra SD card.

We tested the following configurations. (1) *BM*: bare-metal environment, (2) *QEMU*: fully-emulated VM on QEMU, (3) *KVM*: VM

TABLE I
EVALUATED OP-TEE TA

TA	Description
acipher	Generates a 2048-bit RSA key pair and encrypts a supplied "hello_world" string using the key.
aes	Runs an AES encryption and decryption from a TA. The non-secure test application provides the AES key, initial vector, and data.
hello_world	A sample TA to answer a hello command and increase an integer value.
hotp	Generates an HMAC-based One Time Password.
random	Generates a random Universally Unique Identifier (UUID)
secure_storage	Access, output, and manage raw file objects over the OP-TEE secure storage.

on mainline KVM, (4) *kvTZ-KVM*: VM on KVM with kvTZ support that integrates the system register optimization, (5) *pKVM*: CVM on pKVM-KVM, (6) *kvTZ-pKVM*: CVM on pKVM-KVM with kvTZ support. We ran the paravirtualized TF-A, OP-TEE kernel, and Linux in a VM and CVM with two vCPUs, a ramdisk, and a virtio network device. We configured VMs/CVMs with 1057 MB RAM on AVA and 640 MB on RPI3B.

We evaluated bare-metal performance on RPI3B only, but not AVA; AVA uses proprietary firmware that prevents us from installing a customized TF-A required by OP-TEE directly. We did not evaluate CVMs on RPI3B because it equips a GICv2, while KVM-pKVM requires GICv3. Due to the limited hardware RAM available, we were unable to execute OP-TEE on QEMU on RPI3B and did not report the results.

1) *Virtualized TrustZone Performance*: We measured the execution time of six different TrustZone applications [17] (see Table I) on AVA and RPI3B. Each executes a specific TA from a userspace app on the Linux-based REE. We also evaluated the cost of micro-level operations as shown in Table IV. As shown in Tables II and III, QEMU's overhead is the most pronounced in acipher. The benchmark performs complex cryptographic operations on CPUs, resulting in higher performance loss due to binary translation. kvTZ enables native execution on KVM, resulting in much faster performance.

Table II shows that kvTZ-KVM incurs more minor overhead in more complex workloads. For example, kvTZ resulted in less than 5% overhead in acipher compared to the BM counterpart. However, it exhibits much higher overhead than BM and simpler workloads. During execution, the evaluated host applications and TAs output to virtual serial devices; they require VM exits to QEMU for I/O emulation, which incurs high overhead. The extent of the slowdown depends on the number of prints issued by an application. For example, we found that hello_world's prints introduced roughly 50 milliseconds of latency. Furthermore, we found that during benchmark executions, the REE and TEE performed multiple world switches; each includes additional exits caused by trap-and-emulate guest TF-A's system register accesses for context switching EL1 states. For example, we found that aes on the unoptimized kvTZ incurred more than 200 milliseconds of slowdown on RPI3B. Table II shows that kvTZ with optimization significantly reduces this overhead. Similar to kvTZ-KVM, kvTZ-pKVM handles the instruction emulation hypercalls made by CVMs in EL2, and achieves similar performance.

Although kvTZ shows significantly worse performance than BM in the SMC benchmark (as seen in Table III), this overhead was amortized mainly in TA benchmarks (Table II). This is because most execution in those benchmarks does not require hypervisor intervention and can run natively. Conversely, kvTZ outperforms BM in Secure I/O. This performance gain is likely due to the VM's more efficient I/O operations

TABLE II
TA PERFORMANCE ON AVA AND RPI3B (IN MILLISECOND)

TA	AVA			RPI3B	
	QEMU	kvTZ-KVM	kvTZ-pKVM	BM	kvTZ-KVM
acipher	5,159.37	671	691	4,736	4,916.5
aes	266.37	22.08	25.40	199.83	316.87
hello_world	211.94	15.18	20.62	148.83	213.87
hotp	278.94	22.21	25.73	181.37	270.73
random	210.66	14.87	17.70	137.73	190.5
secure_storage	561.26	36.45	39.26	404.13	423.43

TABLE III
MICROBENCHMARK PERFORMANCE ON AVA AND RPI3B (IN MILLISECOND)

TA	AVA			RPI3B	
	QEMU	kvTZ-KVM	kvTZ-pKVM	BM	kvTZ-KVM
TACall	1.09	0.08	0.065	0.16	0.67
SMC	0.26	0.045	0.0345	0.01	0.44
Secure I/O	134.54	8.18	7.11	131.35	77.39

TABLE IV
MICROBENCHMARKS

Benchmark	Description
TACall	Transition from an REE app to the TEE to run a TA and return to the app immediately. Measures a single bidirectional base transition cost of TA invocation.
SMC	The REE Linux invokes a SMC instruction, which transitions to the guest TF-A, then to the OP-TEE kernel without performing any work, and reverses the trip, returning to Linux. Measures a single bidirectional base transition cost of the REE and TEE world switch.
Secure I/O	Measures the cost of reading from and writing data into secure storage.

TABLE V
REE APPLICATION BENCHMARKS

Benchmark	Description
Apache	Apache v2.4.54 server handling 100 concurrent requests from a remote ApacheBench v2.3 client, serving the index.html of the GCC 15.10 manual.
Redis	Redis v7.0.7 server serving workload requests from a remote YCSB client.
Memcached	Memcached v1.6.16 using the memtier benchmark v1.2.3 with its default parameters.
Bonnie++	Bonnie++ v1.0.3e benchmark measuring the latency of sequential reads and writes of a file with 1GB size.

on its in-memory ramdisk compared to the BM's operations on the SD card.

2) *REE Application Performance*: We quantified kvTZ's impact on VM and CVM execution in the REE by measuring the performance of widely used application workloads listed in Table V. Fig. 5 presents the results, normalized against VMs with identical configurations running on mainline KVM. Higher values indicate worse performance.

Unlike vTZ, kvTZ requires no hypervisor instrumentation for VM exit handling, resulting in a modest performance impact to VMs and CVMs. Notably, pKVM's performance overhead is significantly higher in Redis than in KVM. We suspect this is due to Redis being more I/O-bound, as CVMs on pKVM using virtio network incur memory copying overhead from bounce-buffering for shared memory communication.

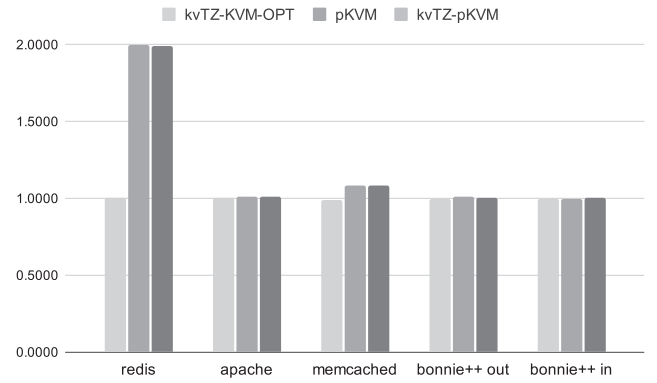


Fig. 5. REE application performance (normalized to KVM).

B. Security Analysis

Existing TEE vulnerabilities, such as CVE-2025-46733 and CVE-2025-38374 from OP-TEE, exploit software bugs to tamper with TEE execution. kvTZ cannot fix or eliminate bugs in TEE software that executes in a virtualized TrustZone.

A malicious guest REE might try to infer or corrupt CPU registers used by the guest TEE or trusted firmware. kvTZ incorporates EL multiplexing, interposing switches between the secure EL3 and REE, as well as between a VM's secure EL1 and EL3 (see Fig. 4), to context-switch EL3 register states, ensuring that their states do not leak to attackers.

A malicious guest REE could also attempt to compromise secure world memory. kvTZ ensures that the REE's stage 2 page table, *normal-VTTBR*, never maps secure memory. Any REE attempts to access secure memory will result in page faults and be rejected. kvTZ traps and emulates MMIO reads and writes to secure virtual devices. It guarantees that guest REE cannot access secure devices. It also ensures that interrupts from secure devices can only be injected into the guest secure world, preventing interrupt hijacking by the REE.

kvTZ's pKVM implementation extends protection to both a CVM's normal world and secure world. An attacker controlling the KVM host might attempt to access or corrupt a CVM's secure world CPU and memory states. kvTZ's pKVM extension interposes CVM exits and saves the CVM's CPU states in pKVM's address space,

inaccessible to the host. Furthermore, pKVM allocates the CVM's page tables (*normal-VTTBR* and *secure-VTTBR*) from its own address space to prevent tampering. pKVM ensures that a CVM's secure memory is unmapped from the hypervisor host's S2PT. We route S2PT page faults to pKVM to reject invalid memory operations. We inherit pKVM's design and offload I/O virtualization, such as semihosting emulation, to KVM. We extended pKVM to expose non-secret I/O values, enabling the use of host functionality while safeguarding the VM's private CPU states.

VI. RELATED WORK

Compared to vTZ [7], kvTZ offers broader compatibility, supporting both legacy VMs and CVMs. For CVMs, kvTZ not only protects the entire VM from the hypervisor—matching vTZ's TEE protection—but also provides substantially stronger, inherent security guarantees for the VM's normal world. Furthermore, kvTZ provides a virtual EL3 environment, enabling VMs and CVMs to execute trusted firmware and leverage its security features.

Similarly, TEEVseL4 [18] virtualized TrustZone to execute guest TEEs. TEEVseL4 takes a different design from vTZ and kvTZ. It employs a microkernel-based approach to TrustZone virtualization by extending the virtualization capabilities of seL4. While it avoids hardware TrustZone dependencies like our approach, TEEVseL4 differs by focusing solely on virtualizing the TrustZone environment for TEE kernels and TAs without supporting trusted firmware execution in VMs. Additionally, both TEEVseL4 and vTZ lack support for exposing virtualized TrustZone to confidential VMs.

Previous research [19], [20] focuses on a different direction from kvTZ. They focus on enabling multiple TrustZone-based TEEs on a single host hardware. TEEv [19] relies on a hypervisor called TEE-visor that runs in the secure world to manage virtual TEE instances that each run a vendor-specific TEE-kernel and TAs. MyTEE [20] offers an alternative approach designed specifically for systems lacking key TrustZone hardware components for memory protection. It incorporates a small hypervisor to manage Arm's S2PTs to achieve memory isolation between TEEs and the host's REE. In future work, we plan to investigate extending KVM to provide isolated TEE environments on the host system.

HA-VMSI [21] and Twinvisor [22] extended TrustZone's secure monitor to utilize hardware features to isolate virtual machines' resources from an untrusted hypervisor host. However, neither work supports TrustZone virtualization. kvTZ extends KVM with pKVM support, maintaining VM protection against the untrusted hypervisor host and enabling virtualized TrustZone in this protected VM environment.

VII. DISCUSSION AND FUTURE WORK

We believe kvTZ applies to Xen [23] to enable virtualized TrustZone. Like KVM, Xen also leverages Arm VE to host VMs. The Xen hypervisor that runs in EL2 deprives the service VM, Dom0, and user VMs, DomU, in less privileged EL0/EL1. We could extend the Xen hypervisor, which handles CPU and memory virtualization, to support EL multiplexing and manage *normal-VTTBR* and *secure-VTTBR*. For the latter, the enlightened Xen handles the ERET and SMC instructions invoked by DomU and multiplex VM states on the normal EL1 hardware accordingly. Xen integrates with QEMU, like KVM, to utilize its virtual devices. We could extend Xen following kvTZ's design to utilize QEMU's TrustZone I/O support.

The current kvTZ implementation supports QEMU's hardware abstraction for TrustZone. QEMU lacks essential TrustZone extensions,

such as the TrustZone Address Space Controller (TZASC) and the TrustZone Memory Adapter (TZMA). TEEs can program these extensions to impose memory isolation. TrustZone provides similar hardware extensions to TEEs to restrict access to secure I/O devices. We plan to extend QEMU to implement these hardware extensions to a guest TEE. Further, we will explore integrating KVM's vIOMMU [24] to support DMA protection features in the virtualized TrustZone. These features could bolster the virtualized environment's isolation boundaries between the secure and normal world. Further, we plan to extend KVM and QEMU for management features such as snapshots and live migration of VMs with a virtualized TrustZone. Users can leverage the enhanced mobility of virtualized TrustZone.

We will explore extending Arm CCA to provide virtualized TrustZone capabilities for CCA-based CVMs. We plan to extend CCA's TCB, Realm Management Monitor (RMM), by mirroring the pKVM implementation strategy to implement EL multiplexing and trap-and-emulate mechanisms for secure world-sensitive instructions while strictly isolating virtualized secure world memory and I/O devices.

VIII. CONCLUSION

We introduced a new kvTZ design that bridges a critical gap by exposing TrustZone to virtualized environments, significantly advancing its security capabilities. We developed a new EL multiplexing mechanism that enables the execution of a full TrustZone software stack, including TEE, and secure firmware. kvTZ's KVM prototype supports OP-TEE on legacy and confidential VMs while preserving performance efficiency on Arm hardware. We have released the kvTZ implementation in: <https://github.com/ntu-ssl/linux-kvtz>.

ACKNOWLEDGMENT

We thank the anonymous reviewers for your insightful feedback to the earlier draft of the paper. We also thank Keng-Lun Chang for contributing to the implementation of memory virtualization for kvTZ.

REFERENCES

- [1] Arm Limited, "TrustZone for Cortex-A," Jan. 2024. [Online]. Available: <https://www.arm.com/technologies/trustzone-for-cortex-a>
- [2] Arm Limited, "Arm trusted firmware," 2022. [Online]. Available: <https://www.trustedfirmware.org/projects/tf-a>
- [3] C. Williams, "Microsoft: Can't wait for ARM to power MOST of our cloud data centers! Take that, Intel! Ha! Ha!," *The Register*, Mar. 2017. [Online]. Available: https://www.theregister.co.uk/2017/03/09/microsoft_arm_server_followup
- [4] Amazon Web Services, "Introducing Amazon EC2 A1 instances powered by new arm-based AWS graviton processors," Nov. 2018. [Online]. Available: <https://aws.amazon.com/about-aws/whats-new/2018/11/introducing-amazon-ec2-a1-instances>
- [5] KVM, (n.d.). [Online]. Available: https://www.linux-kvm.org/page/Main_Page
- [6] Jake Edge, "KVM for android," LWN.net, Nov. 2020. [Online]. Available: <https://lwn.net/Articles/836693/>
- [7] Z. Hua, J. Gu, Y. Xia, H. Chen, B. Zang, and H. Guan, "vTZ: Virtualizing ARM TrustZone," in *Proc. 26th USENIX Secur. Symp.*, 2017, pp. 541–556. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/hua>
- [8] G. A. K. Team, "Android KVM Linux kernel," 2022. [Online]. Available: <https://android-kvm.googlesource.com/linux/+refs/heads/pkvm>
- [9] About OP-TEE, (n.d.). [Online]. Available: <https://optee.readthedocs.io/en/latest/general/about.html>
- [10] Arm Limited, "SMC (secure monitor call)," (n.d.). [Online]. Available: <https://developer.arm.com/documentation/ddi0602/2024-09/Base-Instructions/SMC--Secure-monitor-call>
- [11] A. Kivity, Y. Kamay, D. Laor, U. Lublin, and A. Liguori, "KVM: The Linux virtual machine monitor," in *Proc. 2007 Ottawa Linux Symp.*, 2007, pp. 225–230.

- [12] QEMU, (n.d.). [Online]. Available: <https://www.qemu.org/docs/master/>
- [13] Tiny code generator, (n.d.). [Online]. Available: <https://www.qemu.org/docs/master/devel/index-tcg.html>
- [14] ARM Ltd., "ARM architecture reference manual ARMv8-A DDI0487A.a," Sep. 2013.
- [15] C. Dall and J. Nieh, "KVM/ARM: The design and implementation of the Linux ARM hypervisor," in *Proc. 19th Int. Conf. Architect. Support Program. Lang. Operating Syst.*, 2014, pp. 333–348. [Online]. Available: <https://doi.org/10.1145/2541940.2541946>
- [16] A. T. Inc., "AVA developer platform," 2021. [Online]. Available: <https://www.ipi.wiki/pages/com-hpc-altra>
- [17] TrustedFirmware.org, "OPTEE examples," Nov. 2024. [Online]. Available: https://optee.readthedocs.io/en/latest/building/gits/optee_examples/optee_examples.html
- [18] B. Blazevic, M. Peter, M. Hamad, and S. Steinhorst, "TEEVseL4: Trusted execution environment for virtualized seL4-based systems," in *Proc. IEEE 29th Int. Conf. Embedded Real-Time Comput. Syst. Appl.*, 2023, pp. 67–76.
- [19] W. Li, Y. Xia, L. Lu, H. Chen, and B. Zang, "TEEv: Virtualizing trusted execution environments on mobile platforms," in *Proc. 15th ACM SIGPLAN/SIGOPS Int. Conf. Virtual Execution Environ.*, 2019, pp. 2–16. [Online]. Available: <https://doi.org/10.1145/3313808.3313810>
- [20] S.-K. Han and J. Jang, "MyTEE: Own the trusted execution environment on embedded devices," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023.
- [21] M. Zhu, B. Tu, W. Wei, and D. Meng, "HA-VMSE: A lightweight virtual machine isolation approach with commodity hardware for ARM," in *Proc. 13th ACM SIGPLAN/SIGOPS Int. Conf. Virtual Execution Environ.*, 2017, pp. 242–256.
- [22] D. Li, Z. Mi, Y. Xia, B. Zang, H. Chen, and H. Guan, "TwinVisor: Hardware-isolated confidential virtual machines for ARM," in *Proc. ACM SIGOPS 28th Symp. Operating Syst. Princ.*, 2021, pp. 638–654.
- [23] P. Barham et al., "Xen and the art of virtualization," in *Proc. 19th ACM Symp. Operating Syst. Princ.*, 2003, pp. 164–177.
- [24] N. Amit, M. Ben-Yehuda, I. Research, D. Tsafir, and A. Schuster, "vIOMMU: Efficient IOMMU emulation," in *Proc. 2011 USENIX Annu. Tech. Conf.*, 2011, Art. no. 6. [Online]. Available: <https://www.usenix.org/conference/usenixatc11/viommu-efficient-iommu-emulation>
- [25] ARM Ltd., "Introducing ARM confidential compute architecture version 1," May 2022. [Online]. Available: <https://developer.arm.com/documentation/den0125/0100/What-is-Arm-CCA->
- [26] P. Elwell et al., "Linux kernel source tree for Raspberry Pi," 2025. [Online]. Available: <https://github.com/raspberrypi/linux>